Лекція №18 (2 години). Навігація в SPA: React Router.

План лекції

1. Проблема маршрутизації (Routing) у Single Page Applications.
2. Встановлення та базова конфігурація React Router DOM (v6+).
3. Створення маршрутів (`Routes` та `Route`).
4. Навігація між сторінками: компонент `<Link>` vs тег `<a>` .
5. Динамічні параметри маршрутів (URL Parameters) та хук `useParams` .
6. Вкладені маршрути (Nested Routes) та компонент `<Outlet>` .
7. Програмна навігація: хук `useNavigate` .
8. Обробка помилок `404 Not Found` .

Перелік умовних скорочень

- **SPA** (Single Page Application) — односторінковий додаток.
- **URL** (Uniform Resource Locator) — уніфікований покажчик ресурсу (адреса).
- **DOM** (Document Object Model) — об'єктна модель документа.
- **Router** — маршрутизатор.

Вступ

З попередніх лекцій ми знаємо, що React створює **Односторінкові Додатки (SPA)**. Це означає, що фізично на сервері лежить лише один файл `index.html` . Але ж користувачам звично бачити різні "сторінки": Головну, "Про нас", "Контакти", Сторінку конкретного товару. Вони очікують, що при переході на нову "сторінку" адреса в їхньому браузері (URL) зміниться з `mysite.com/` на `mysite.com/about` , і вони зможуть скопіювати це посилання та надіслати другу або додати в закладки.

Оскільки браузер більше не робить запитів до сервера за новими сторінками після кожного кліку (в цьому вся суть SPA), нам потрібен спеціальний механізм. Цей механізм буде **перехоплювати зміну URL-адреси** у браузері (за допомогою History API) і змушувати React **малювати інший Компонент** на екрані, імітуючи зміну сторінок.

Цей механізм називається Клієнтська Маршрутизація (Client-side Routing). Оскільки React — це лише бібліотека для UI (як ми вивчили у Лекції 14), маршрутизатора у нього "з коробки" немає. Індустріальним стандартом де-факто для вирішення цієї проблеми стала бібліотека **React Router**.

Мета цієї лекції — навчитися створювати багаторівневу структуру "сторінок" нашого додатку та мандрувати між ними без перезавантаження браузера.

1. Встановлення та базова конфігурація

React Router — це окрема бібліотека, тому її потрібно встановити через npm у ваш проект:

```
npm install react-router-dom
```

Для того, щоб Router запрацював, ВЕСЬ ваш додаток має бути огорнутий у спеціальний компонент-провайдер `<BrowserRouter>`. Традиційно це роблять у найголовнішому файлі `main.jsx` (або `index.js`).

```
// файл src/main.jsx
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom"; // 1. Імпортуємо
import App from "./App";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <BrowserRouter>
      {" "}
      {/* 2. Огортаємо App */}
      <App />
    </BrowserRouter>
  </React.StrictMode>,
);
```

2. Створення маршрутів (`Routes` та `Route`)

У нашому головному файлі (зазвичай `App.jsx`) ми визначаємо правила дорожнього руху нашого сайту. Ми кажемо: "*Якщо в адресному рядку браузера написано X, покажи компонент Y*".

Для цього використовуються два компоненти:

- `<Routes>` : Контейнер, який переглядає всі свої дочірні маршрути і обирає ТОЙ ЄДИНИЙ, який зараз найкраще відповідає поточному URL.
- `<Route>` : Сам маршрут. Має два обов'язкові пропси: `path` (який URL чекаємо) та `element` (який компонент хочемо намалювати).

```
// файл src/App.jsx
import { Routes, Route } from "react-router-dom";
import Home from "./pages/Home";
import About from "./pages/About";
import Contact from "./pages/Contact";

function App() {
  return (
    <div>
      {/* Меню сайту буде показуватися на КОЖНІЙ сторінці, бо воно поза <Routes>
*/}
      <nav>Наше Супер Меню</nav>

      {/* Тут буде "підставлятися" потрібна сторінка */}
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </div>
  );
}
```

3. Навігація: компонент `<Link>` проти тегу `<a>`

Якщо ми спробуємо створити навігаційне меню за допомогою стандартного тегу `<a>`, наприклад `<a href="/about">Про нас</a>`, додаток **зламається як SPA**. Чому? Тому що браузер побачить посилання і по звичці відправить "жорсткий" запит на сервер за файлом `/about.html`, якого не існує. Сторінка повністю перезавантажиться (білий екран).

В React Router для мандрування **ЗАБОРОНЕНО** використовувати `<a>` для внутрішніх лінків ресурсу. Ми зобов'язані імпортувати і використовувати компонент `<Link>`.

```
import { Link } from "react-router-dom";

function Navigation() {
  return (
    <nav>
      {/* Замість href пишемо to */}
      <Link to="/">Головна</Link>
      <Link to="/about">Про нас</Link>
      <Link to="/contact">Контакти</Link>

      {/* Тег 'a' можна використовувати лише для сайтів ВАШИХ конкурентів або
соцмереж */}
      <a href="https://google.com">Зовнішній ресурс Google</a>
    </nav>
  );
}
```

Під капотом `<Link>` не дозволяє браузеру звертатися до сервера (через `event.preventDefault()`). Він просто змінює URL в адресному рядку, Router бачить зміну і миттєво підміняє компонент у `<Routes>`.

> *Також є компонент `<NavLink>`, який вміє автоматично додавати CSS-клас `active` (або інший), якщо ви зараз знаходитесь на цій сторінці (дуже зручно для підсвітки пунктів меню).*

4. Динамічні параметри (URL Parameters)

Уявіть інтернет магазин, де є 10 000 товарів. Не будемо ж ми вручну писати 10 000 тегів `<Route>`. Нам потрібен один **динамічний** маршрут. У `path` ми ставимо двокрапку `:` перед назвою змінної, щоб сигналізувати роутеру, що тут може бути будь-яке число або текст.

```
{
  /* App.jsx */
}
{
  /* Злови URL типу /products/7 або /products/apple */
}
<Route path="/products/:id" element={<ProductDetails />} />;
```

Як всередині компонента `ProductDetails` дізнатися, по якому саме товару клікнули (чи це 7, чи apple)? Для цього використовується спеціальний **хук** `useParams`:

```
// файл ProductDetails.jsx
import { useParams } from "react-router-dom";

function ProductDetails() {
  // Хук useParams повертає об'єкт з усіма параметрами з URL
  const { id } = useParams();

  // Тепер ми можемо використати цей id для useEffect(fetch), щоб дістати цей
товар з бази
  return <h1>Ви переглядаєте товар з ідентифікатором: {id}</h1>;
}
```

5. Вкладені маршрути (Nested Routes) та `<Outlet>`

У великих додатках дизайни часто вкладені один в одного. Наприклад, в Адмін-панелі сайту завжди є лівий Sidebar з навігацією. Якщо ми перейдемо до списку користувачів (`/admin/users`) або до налаштувань (`/admin/settings`), Sidebar має залишатися на місці. Змінюється лише права частина екрану.

Це і є Вкладений Роутінг. Маршрут-Батько містить всередині інші Маршрути-Діти. Робиться це шляхом розміщення тегів `<Route>` всередині іншого `<Route>`.

```
{
  /* App.jsx */
}
<Routes>
  <Route path="/" element={<Home />} />

  {/* Батьківський маршрут /admin */}
  <Route path="/admin" element={<AdminLayout />}>
    {/* Дочірні маршрути. Їхні шляхи сумуються з батьківськими */}
    <Route path="users" element={<UsersPanel />} /> {/* url: /admin/users    */}
    <Route path="settings" element={<SettingsTab />} />{" "}
    {/* url: /admin/settings */}
  </Route>
</Routes>;
```

Для того, щоб Батьківський компонент `AdminLayout` знав, у якому саме своєму місці він повинен намалювати Дітей, йому потрібно дати спеціальне маркування — компонент **`<Outlet/>`** (Вихідне сопло/Дірка).

```
import { Outlet, Link } from "react-router-dom";

function AdminLayout() {
  return (
    <div style={{ display: "flex" }}>
      <aside className="sidebar">
        <Link to="/admin/users">Користувачі</Link>
        <Link to="/admin/settings">Налаштування</Link>
      </aside>

      <main className="content">
        {/* Саме сюди будуть проектуватися компоненти UsersPanel або SettingsTab!
*/}
        <Outlet />
      </main>
    </div>
  );
}
```

6. Програмна навігація: хук `useNavigate`

`Link` чудово підходить для кліків по меню. Але часто нам потрібно перенаправити користувача "непомітно" у відповідь на логіку коду. Наприклад: "Після того, як форма була успішно відправлена на сервер, перекинь користувача на сторінку /success". Тут немає кліку по лінку. Нам потрібна команда з коду — Програмна навігація (Programmatic Navigation).

Для цього використовується **хук** `useNavigate`, який повертає функцію, яку ви можете викликати де завгодно.

```
import { useNavigate } from "react-router-dom";

function LoginForm() {
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
```

```
    // 1. Уявімо, ми відправляємо логін і пароль на сервер
    const success = await loginUserAPI();

    if (success) {
      // 2. Якщо все добре, примусово перекидаємо юзера в Дашборд
      navigate("/dashboard");
    } else {
      alert("Невірний пароль");
    }
  };

  return <form onSubmit={handleSubmit}>...</form>;
}
```

7. Обробка помилок `404 Not Found`

Якщо користувач введе в браузер неіснуючу адресу (наприклад, `mysite.com/hocus-pocus`), жоден з наших маршрутів `<Route>` не відреагує. Екран буде абсолютно порожнім. Це дуже поганий UX.

Щоб "перехопити" будь-яке невідоме посилання, в самому кінці списку `<Routes>` потрібно поставити маршрут зі шляхом Зірочка `*` (універсальний селектор "все інше").

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<About />} />

  {/* Якщо URL не підійшов ні під Home, ні під About - спрацює цей: */}
  <Route path="*" element={<PageNotFound />} />
</Routes>
```

У компоненті `PageNotFound` зазвичай малюють велику цифру 404, кота, і кнопку `Повернутися на Головну`.

Висновки

1. Завдяки Клієнтській Маршрутизації (React Router) ми створюємо ілюзію багаторівневого сайту в межах Single Page Application (SPA) без жодного перезавантаження браузера.

2. Весь проект має бути огорнутим у `<BrowserRouter>`. Для налаштування "шляхів" використовують пару `<Routes>` і `<Route path="..." element={...}>`.
3. Забути про тег `<a href="..">`! Всередині вашого додатку всі посилання мають робитися через компонент `<Link to="...">`, інакше браузер висмикне вас із концепції SPA і піде на сервер.
4. Символ двокрапки `:` у властивості `path="/user/:id"` створює динамічний маршрут. Зловити це конкретне значення параметра всередині компонента можна за допомогою хука `useParams()`.
5. Хук `useNavigate()` — інструмент для автоматичного (програмного) перекидання користувача після виконання якихось логічних операцій (наприклад, реєстрації чи натискання кнопки-скасувати).
6. Вкладена маршрутизація та компонент-вказівник `<Outlet/>` дозволяють створювати складні інтерфейси із "фіксованими каркасами" (наприклад, адмін панель), в яких перемальовується лише центральна частина.
7. Маршрут із властивістю `path="*"` (зірочка) в кінці списку Routes — обов'язкова практика створення безпечного екрану 404 Помилки (Сторінку Не Знайдено).

Джерела

1. React Router Documentation: Tutorial — Офіційний покроковий туторіал від розробників бібліотеки. Повчально і суворо.
2. UI.dev: React Router v6 in 20 Minutes

Запитання для самоперевірки

1. Чому React Router є сторонньою бібліотекою, а не вбудований у React? Що таке клієнтська маршрутизація?
2. Поясніть шкоду, яку спричинить тег `<a>` замість `<Link>` для навігації в межах SPA-додатку.
3. Що означає двокрапка `:` у рядку `/product/:id` під час налаштування маршруту? Який вбудований хук React Router допоможе компоненту витягти значення на місці цього двокрапки з адреси браузера?
4. В якій ситуації і для якої поведінки вам би довелося застосувати хук `useNavigate()` замість `<Link>`?
5. Для чого на сторінці-шаблоні Батьківського Маршруту (Parent Layout) потрібно вставляти компонент `<Outlet />`? Під час розробки яких поширених UI-шаблонів це корисно?
6. Як налаштувати в React Router обробку невідомих маршрутів, щоб створити сторінку "404 Not Found" (Який "символ" відповідає за перехоплення всіх неспівпадінь)?